DAY 38

Model Saving, Loading & Deployment Basics

60-Day Data Science Learning Journey

A practical, beginner-friendly guide to training a model, keeping preprocessing consistent, saving the trained pipeline, loading it again, and predicting on new data.

What you will build A complete Scikit-learn workflow that trains, evaluates, saves, reloads, and reuses a model on a fictional dataset.
Main tools Python, pandas, scikit-learn, joblib
Important limitation The dataset is synthetic and the project is for learning only, not for real academic decisions.

Prepared as a study resource with commented Python practice and revision tasks.

1. Learning objectives

By the end of Day 38, you should be able to:

• Explain the difference between model training and prediction.

• Use `joblib.dump()` to save a trained model and `joblib.load()` to load it.

• Keep preprocessing and the estimator together in a Scikit-learn Pipeline.

• Evaluate on held-out test data before treating a model as useful.

• Make predictions for new examples with the same feature schema.

• Recognize security, reproducibility, fairness, and monitoring concerns.

2. The workflow in plain language

Step	Purpose
1. Prepare data	Choose input features and target labels.
2. Split data	Keep a test set for an honest evaluation.
3. Build a pipeline	Bundle transformations and model together.
4. Train and evaluate	Use fit on training data and evaluate on held-out data.
5. Save and load	Serialize the trusted trained pipeline and reload it later.
6. Predict	Pass new inputs with the same columns and compatible types.

3. Core concepts

Training versus prediction

Training: the model learns patterns from examples using `fit(X_train, y_train)`. **Prediction:** the trained model produces outputs for new input rows using `predict(X_new)`.

What is a Pipeline?

A Pipeline chains preprocessing and a model into one object. This reduces the risk of applying one transformation during training and a different transformation during prediction. In this lesson, StandardScaler standardizes numeric features before Logistic Regression.

Why save the whole pipeline?

Saving only the classifier while forgetting the scaler can produce inconsistent predictions. Saving the pipeline preserves both the fitted scaler and fitted classifier.

What does joblib do?

```
import joblib

joblib.dump(model, "model.joblib")       # save
loaded_model = joblib.load("model.joblib")  # load
prediction = loaded_model.predict(new_data)
```

Only load serialized files from sources you trust. Serialized Python objects can be dangerous when they come from unknown sources.

4. Complete project code

The full runnable script is provided separately as **Day-38-Model-Saving-Loading-Deployment.py**. The essential workflow is reproduced below for study.

```
import pandas as pd
import joblib
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

data = {
    "study_hours": [1,2,2,3,3,4,4,5,5,6,6,7,7,8,8,9],
    "attendance":  [40,50,55,45,60,55,65,60,70,65,75,70,80,75,90,95],
    "result": [
        "Needs Support","Needs Support","Needs Support","Needs Support",
        "Needs Support","Needs Support","Pass","Needs Support",
        "Pass","Pass","Pass","Pass","Pass","Pass","Pass","Pass"
    ]
}
df = pd.DataFrame(data)
X = df[["study_hours", "attendance"]]
y = df["result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000, random_state=42))
])
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions, zero_division=0))

joblib.dump(model, "student_result_model.joblib")
loaded_model = joblib.load("student_result_model.joblib")

new_student = pd.DataFrame({
    "study_hours": [6],
    "attendance": [80]
})
print("Prediction:", loaded_model.predict(new_student)[0])
```

Code walkthrough

• X contains the input features. y contains the target label.

• `train_test_split()` keeps a held-out portion for evaluation. Stratification helps preserve class proportions in this small dataset.

• `Pipeline()` ensures scaling is learned only from training data and is reused consistently.

• `fit()` trains the pipeline. The test set is not used for fitting.

• `classification_report()` gives precision, recall, F1-score, and support by class.

• `joblib.dump()` saves the fitted pipeline. `joblib.load()` restores it.

• The new input is a DataFrame with the same feature names and order as training.

5. Understanding evaluation

Accuracy is the fraction of test predictions that are correct. It can be misleading when classes are imbalanced. Precision, recall, and F1-score provide additional information. A tiny synthetic dataset produces unstable metrics,

so use it to learn the workflow rather than to judge a real application.

Never report test performance as a guarantee of future performance. Real deployment requires representative validation data, monitoring, and regular review.

6. Common mistakes and fixes

Common mistake	Better practice
Saving the wrong object	Save the fitted pipeline, not an unfitted model or only a preprocessing step.
Preprocessing mismatch	Bundle transformations and estimator in a Pipeline.
Training on test data	Fit only on training data. Use the test set for final evaluation.
Different input columns	Pass the same feature names and compatible types as used during training.
Untrusted model files	Never load joblib/pickle files from unknown sources.
Version incompatibility	Record compatible Python and library versions for reproducibility.
No monitoring	Monitor input quality, changing patterns, failures, and performance where labels become available.

7. Practice tasks

Task 1: Change the new example to 4 study hours and 70 attendance. Make a prediction.

Task 2: Print `model.classes_` and explain the order of the class probability values.

Task 3: Compare predictions from the in-memory model and the loaded model for the same input.

Task 4: Explain why a Pipeline is safer than manually scaling new input separately.

Task 5: Name two risks of using a model in a real application.

Suggested answers

1. Change the values in the `new_student` DataFrame and call `predict()`.

2. `classes_` lists the label order. Each probability in `predict_proba()` corresponds to that order.

3. Call `predict()` on both models and compare the returned labels. For the same saved fitted pipeline and same input, they should match.

4. A Pipeline reuses the fitted preprocessing parameters and reduces training/prediction mismatch.

5. Examples include poor data quality, bias, changing data patterns, unsafe model files, privacy concerns, and over-reliance on predictions.

8. Quick revision checklist

■ I can explain fit() versus predict().

■ I know how to save and load a trusted model using joblib.

■ I understand why preprocessing should be included in a Pipeline.

■ I know the test set should not be used to train the model.

■ I can make predictions using the same input features.

■ I understand that saved models still need security, evaluation, and monitoring.

9. LinkedIn post

A separate copy-ready text file is included. It summarizes the key ideas from Day 38 and contains the requested GitHub repository link.

GitHub: https://github.com/DeveloperDesigner-star/Datasciencepdfs

Responsible-use reminder

This project uses a very small fictional dataset and is for learning only. Do not use it to make real decisions about students. The code was syntax-checked, but the full ML workflow was not runtime-tested during file generation.